



pypi package or version not found python 3.12+ license Apache 2.0

CLI + Template to scaffold and ship AI agent projects (FastAPI + React/Vite/TS)

Description

synapseForge is a PyPI package that provides a CLI to scaffold full-stack AI agent projects from scratch — backend (FastAPI), frontend (React/Vite/TypeScript), branding (logo, .ico, color palette), dependencies, and a self-contained distribution build.

The generated project includes:

- **Agent Framework** (backend/agent/): AgentLoop, Tools Registry (native + external + MCP), Sessions (SQLite WAL), Permissions, Skills, MCP integration
- **Context Files**: Upload PDF, Word, TXT, MD, CSV, JSON, YAML, XML, PY — text extraction (pdfminer + OCR fallback) with automatic injection into system prompt
- **Metrics**: Aggregated usage stats, per-session breakdown, token usage by model/provider
- **File Extraction**: Shared module supporting plain text, Markdown, CSV, JSON, XML, YAML, Python, DOCX, DOC (via LibreOffice), XLSX, XLS (via LibreOffice), PDF (pdfminer + optional Tesseract OCR)
- **Frontend** (React/Vite/TS): Chat SSE streaming, config panel (provider/model/context), sessions sidebar, context files management, tool calls visualization, metrics dashboard
- **User Config** (~/.config/synapseForge/): Custom tools, skills, agent definitions with permissions, MCP server config
- **Docker**: Multi-stage Dockerfile + docker-compose.yml for single-container deployment
- **Desktop App Mode**: Heartbeat watchdog (3min timeout → auto-exit), shutdown endpoint, `synapseforge run` for dev

Installation

```
pip install synapseforge
```

Package dependencies (only these two — everything else is project-level):

- `colorthief>=0.2.0` — Automatic color palette extraction from the logo
- `Pillow>=10.0.0` — .ico favicon generation

Quick Start

Create a new project

```
synapseforge init my-project
```

Interactive GUI pipeline (3 tabs):

1. **Project** — Company logo (for README), client logo (for app, optional), company name, owner, legal name, repo name, client name, description, task (all required).
2. **Logos** — File pickers for both logos, optional width/height for company logo.



3. **Colors** — 8 optional hex fields with color picker: Avatar asistente, Avatar usuario, Botón Nuevo Chat fondo/texto, Botón adjuntar, Botón enviar, Botón detener, Flecha autoscroll. Leave empty to auto-extract from client logo.

```
cd my-project
# Ready to develop
```

Target directory behavior:

- If it **does not exist** → creates it and extracts template.
- If it **exists and is empty** → extracts template cleanly.
- If it **exists with files** → extracts on top. Zip files overwrite. Pre-existing files not in zip **remain** (orphaned).

Build a distributable

```
synapseforge launch ./my-project "MyApp"
```

1. **Frontend** — npm run build with VITE_MODE=prod → frontend/dist/
2. **Backend** — Compile to .pyc , clean copy (exclude .py , __pycache__ , sessions.db , docs outside agent/prompts/)
3. **Embedded Python** — Downloads Python 3.12.0 embed amd64, configures _pth , installs pip, installs requirements.txt deps
4. **Launcher** — Generates customized launcher.py → PyInstaller --onefile --noconsole → MyApp.exe
5. **Package** — Bundles MyApp.exe , backend/ , frontend/dist/ , python/ , .env , LICENSE , README.md into a timestamped zip
6. **Cleanup** — Moves zip to project root, removes build artifacts, cleans .pyc from original backend

Edit colors at runtime (no rebuild)

```
synapseforge colors ./my-project
```

Opens tkinter GUI with 8 color fields + live preview squares + native color picker. Saves to frontend/public/colors.json . **Refresh browser (F5)** to see changes instantly.

Run development servers

```
synapseforge run ./my-project
```

Starts uvicorn backend.main:app --reload --port 8000 + npm run dev in frontend/ , waits 3s, opens http://localhost:5173 . **Ctrl+C stops both.**

Requirements

Tool	Version	Needed for
Python	3.12+	init, launch, run, colors
Node.js	20+	launch (frontend build), run (dev server)
Docker	20+	Optional: containerized deployment

Generated project structure

The structure created by synapseforge init (the template embedded in pipeline/template.zip):



```
my-project/
├── backend/
│   ├── __init__.py
│   ├── main.py                # FastAPI app (CORS, lifespan, routers, health, shutdown, heartbeat, SPA static)
│   ├── instances.py          # Singletons: agent, session_manager
│   ├── routes/
│   │   ├── __init__.py
│   │   ├── chat.py           # POST /api/chat → SSE stream (AgentLoop)
│   │   ├── config.py         # Providers, models, MCP health, context window
│   │   ├── sessions.py       # CRUD sessions, messages, titles
│   │   ├── context_files.py  # CRUD context files (instructions/documents)
│   │   ├── metrics.py        # Metrics: aggregated, per-session, tokens by model/provider
│   │   ├── file_text_extractor.py # Text extraction: PDF, DOCX, XLSX, TXT + OCR fallback
│   │   └── README.md
│   └── agent/                # Agent framework (Loop, Tools, MCP, Skills, Sessions, Permissions)
│       ├── __init__.py
│       ├── agent.py          # Agent class (Groq/Ollama, streaming SSE, tool calling)
│       ├── tools.py          # Registry: native + external + MCP
│       ├── loop.py           # Agentloop: while True → LLM → tool_calls → execute → continue
│       ├── loop_helpers.py   # build_system_prompt (injects context_files), fetch_context_window, execute_tool
│       ├── session.py        # SessionManager (SQLite WAL, history, config_kv, error_log)
│       ├── permissions.py    # Permissions per agent (tool/skill/task allow/deny/ask + wildcards)
│       ├── config_dir.py     # ~/.config/synapseForge/ discovery
│       ├── contract.py       # ContractResponse, UsageReport, StreamingResponse
│       ├── ddl_setup.py      # SQLite table initialization
│       ├── agent_db/        # SQLite runtime (created on start)
│       ├── prompts/
│       │   ├── system_prompt.md # Router base prompt (with fidelity checklist)
│       │   ├── help.md        # Internal documentation for `help` tool
│       │   └── title.md       # Prompt for session title generation
│       ├── utils/
│       │   ├── __init__.py
│       │   ├── clean_memory.py # GPU/CPU model release
│       │   ├── model_resolver.py # Active model resolution and validation
│       │   ├── skill_loader.py # SKILL.md loading and formatting for system prompt
│       │   ├── email_parser.py # Email parsing (headers, body, attachments)
│       │   ├── mcp_helper.py   # MCP stdio/HTTP, tool discovery, health check
│       │   ├── subagent_logger.py # Custom SUBAGENT log level
│       │   └── error_logger.py # Error logging to SQLite (error_log table)
│       └── README.md
├── frontend/
│   ├── package.json
│   ├── package-lock.json
│   ├── tsconfig.json
│   ├── tsconfig.app.json
│   ├── tsconfig.node.json
│   ├── vite.config.ts
│   ├── index.html
│   ├── public/
│   │   └── docs.html
│   └── src/
│       ├── main.tsx          # Entry: loads colors.json → sets CSS vars → render App
│       ├── App.tsx           # Root + providers
│       └── index.css          # @theme Tailwind v4: 8 configurable vars + fixed
```



```
|   |   | vite-env.d.ts
|   |   | assets/
|   |   |   | logo_cliente.png
|   |   | services/
|   |   |   | chatService.ts          # SSE parsing: chunk, tool_call, done
|   |   |   | configService.ts        # Providers, models, MCP health
|   |   |   | sessionService.ts        # Sessions CRUD
|   |   |   | contextFilesService.ts   # CRUD context files
|   |   |   | metricsService.ts        # Metrics: overview, sessions, tokens
|   |   |   |   | quoteHistoryService.ts
|   |   | components/
|   |   |   | ChatInterface.tsx
|   |   |   | Sidebar.tsx              # Sessions + Config tabs
|   |   |   | MessageBubble.tsx
|   |   |   | MetricsModal.tsx         # Metrics dashboard
|   |   |   | HistoryModal.tsx
|   |   |   | Logo.tsx
|   |   |   |   | ui/                  # shadcn/ui: avatar, button, dialog, input, separator, textarea, utils
|   |   |   |   | README.md
|   | config/
|   |   | replace.json                # Project placeholders (build-time)
|   | store/                          # Store of installable tools and skills
|   |   | tools_store/
|   |   |   | skills_store/
|   | docs/                           # Product documentation
|   |   | tools/                      # Tool creation guide
|   |   |   | agents/                 # Agent creation guide
|   | src/
|   |   | logo_empresa.png            # Company logo (for README, copied by pipeline)
|   | on_boarding/                    # Developer onboarding
|   |   | ONBOARDING.md
|   |   | CONTRIBUTING.md
|   |   |   | GIT_WORKFLOW.md
|   | cicd/                          # CI/CD
|   | client_db/                     # Client database (template)
|   | tests/                         # Tests
|   | .commands/                     # Local PowerShell commands
|   |   | README.md
|   |   | commands.json
|   |   | init.ps1
|   |   |   | commands/
|   |   |   |   | list_cmds.py
|   |   |   |   | quick_push.py
|   |   |   |   | quick_sync.py
|   | .github/                       # Workflows and PR template
|   |   | PULL_REQUEST_TEMPLATE/
|   |   |   | feature.md
|   |   |   | fix.md
|   |   |   | general.md
|   | Dockerfile                     # Multi-stage: Node build → Python runtime
|   | docker-compose.yml             # Single service, port 8000, VITE_MODE=prod
|   | .dockerignore
|   | requirements.txt
|   | .env.example
```



```
├─ .gitignore
├─ LICENSE
└─ README.md
```

User configuration (`~/ .config/synapseForge/`)

```
~/ .config/synapseForge/
├─ skills/           # Installed skills (SKILL.md per skill + references/)
├─ tools/            # Custom tools (.py files with TOOL_NAME, execute())
├─ agents/           # Agent definitions (.md with YAML frontmatter + permissions + prompt)
└─ config.json       # MCP server configuration
```

Agent definition (`.md` in `agents/`)

```
---
name: "Mi Agente"
description: "Qué hace y cuándo usarlo"
permission:
  read: allow
  write: allow
  task:
    explorador: allow
  skill:
    mi_skill: allow
parameters:
  temperature: 0.0
  top_p: 0.5
  model: null
---
# System prompt del agente (cuerpo del markdown)
```

Permissions: `allow` / `deny` / `ask` (default: `deny`). Supports wildcards (`*`). Nested for `task` (sub-agent names) and `skill` (skill names).

Context handling (planned)

Current state: Context is managed only via **context window** (`context_window_turns`), which limits how many conversation turns are passed to the LLM (`-1` = all history). Automatic compaction/summarization is not implemented — planned for a future release.

Context Files (Instructions & Documents): Files uploaded via Config panel → `POST /api/context-files` → text extracted → concatenated and injected into system prompt for router and sub-agents. Useful for permanent reference: company manuals, business rules, technical docs.

CLI Reference

Command	Description
<code>synapseforge init [dir]</code>	Scaffold a project from bundled template (GUI)
<code>synapseforge launch <path> <exe></code>	Build self-contained distribution zip
<code>synapseforge colors [dir]</code>	Edit frontend/public/colors.json via GUI (live reload)
<code>synapseforge run [dir]</code>	Start uvicorn + npm dev servers, open browser



<code>synapseforge --help</code>	Show global help
----------------------------------	------------------

Tech Stack



License

Apache 2.0

Copyright (c) 2026 SYNASPE AI SAS